

SENTIO

AI-Powered Audience Engagement Platform

About the Project

Sentio is an AI-powered audience engagement platform designed to make live presentations, classroom lectures, and corporate events interactive and engaging. During presentations, audiences often remain passive because people feel hesitant to ask questions or give feedback publicly. Presenters also find it difficult to know if the audience is understanding the content in real time. Sentio solves this problem by allowing audience members to join live sessions from their smartphones or laptops without installing any app or registering an account.

Existing tools like Mentimeter and Slido offer basic live polling and Q&A, but they require presenters to spend a lot of time manually writing poll options, framing quiz questions, and analyzing post-session feedback. Sentio improves this process by integrating Artificial Intelligence. Using an AI service layer powered by Groq, Sentio automatically generates context-aware quizzes and polls based on presentation topics. After a session ends, the AI synthesizes audience responses and generates an executive summary report with key insights and engagement metrics.

I selected this project for my MCA final year because it combines web development, real-time communication, and practical AI application into a single real-world platform.

The expected outcome is a web application where presenters can build interactive slides, start live sessions with 6-character join codes or QR codes, collect real-time responses, and generate AI insights.

Technologies Used:

- **Frontend:** Next.js 16 (React 19), TypeScript, Tailwind CSS
 - **Backend:** Node.js, Express 5, Socket.IO
 - **Database:** MongoDB Atlas with Mongoose
 - **Cloud Storage:** Azure Blob Storage
 - **Email Service:** Resend API
 - **AI Engine:** Groq API (LLM Integration)
-

06/07/2026

Activity

Topic Selection and Preliminary Discussion

Work Completed

Today I spent time brainstorming ideas for my final year MCA project. I evaluated various web application domains and decided to build an interactive audience engagement platform. I looked at existing platforms like Mentimeter and Slido to understand their features and shortcomings. After analyzing their limitations, I conceptualized Sentio, which incorporates AI to automate question generation and session summaries.

Problems Faced

I was unsure if building a real-time system with AI integration would be feasible within the given timeframe for the final year project.

Solution

I discussed the project scope with my classmates and broke down the core requirements into distinct modules so that basic features can be built first before adding advanced AI capabilities.

Outcome

Finalized the project title "Sentio 💎 AI-Powered Audience Engagement Platform" and prepared initial notes for guide approval.

09/07/2026

Activity

Project Guide Approval and Architecture Review

Work Completed

I met with my project guide today to present the project proposal for Sentio. I explained the problem statement, objectives, technology stack, and how AI integration sets it apart from traditional polling platforms. I also presented a preliminary architecture diagram showing how Next.js, Express, Socket.IO, MongoDB, and Groq AI will work together.

Problems Faced

My guide expressed concern regarding potential latency issues when querying AI models during live sessions.

Solution

I explained that live voting and response visualization will run over low-latency WebSockets (Socket.IO), while AI prompt generation will happen asynchronously during slide creation or after the session ends.

Outcome

My project guide approved the proposal and instructed me to prepare the official synopsis for submission.

10/07/2026

Activity

Synopsis Submission and System Architecture Planning

Work Completed

I completed the official project synopsis document today and submitted it to the MCA department. After submission, I spent time planning the monorepo folder structure and refining the system architecture. I decided to use npm workspaces to keep the frontend, backend, and shared TypeScript types organized in a single repository.

Problems Faced

Choosing between a monolithic repository and a monorepo setup took some research to understand how code sharing works across workspaces.

Solution

I reviewed Next.js monorepo documentation and confirmed that npm workspaces with a shared package (`@sentio/shared`) would prevent code duplication between client and server.

Outcome

Synopsis submitted successfully, and the technical architecture and project stack were finalized.

11/07/2026

Activity

Domain Registration, Git Setup, and Project Initialization

Work Completed

Today I purchased the domain **project-sentio.in** for hosting our live presentation platform. I configured the DNS records to point to our deployment services. I created a private GitHub repository, set up `.gitignore`, and initialized the monorepo workspace. I created the basic Next.js app in `apps/frontend` and the Express app in `apps/backend`.

Problems Faced

Configuring domain DNS records took some time to propagate and

verify with external services.

Solution

I verified the DNS settings using online lookup tools and added standard CNAME and ALIAS records for Vercel and Render services.

Outcome

Domain registered, GitHub repository configured, and baseline workspace folders initialized.

12/07/2026

Activity

Dependency Installation and Design System Configuration

Work Completed

Today I installed core project dependencies across the monorepo workspaces. In the frontend app, I configured Tailwind CSS for styling and set up shadcn/ui component utilities. I created global CSS custom properties in `globals.css` for light and dark themes. I also organized the frontend directory structure with dedicated folders for components, pages, hooks, and types.

Problems Faced

Tailwind CSS 4 setup had minor compilation warnings with custom CSS variables when compiling Next.js.

Solution

I updated `globals.css` to use proper `@import "tailwindcss";` and `@theme inline` rules matching Next.js 16 specifications.

Outcome

Frontend styling pipeline configured with reusable UI tokens and organized workspace directories.

13/07/2026

Activity

Database Design and Schema Planning

Work Completed

I spent today designing the MongoDB database schema for Sentio. I mapped out the entity relationships between users, presentations, slides, and notifications. I created the initial Mongoose models for `User`, `Presentation`, `Slide`, and `Notification` in the backend project. I also planned the authentication flow using short-lived JWT access tokens and HTTP-only refresh cookies.

Problems Faced

Designing flexible slide schemas that can accommodate different slide types (polls, quizzes, titles) without bloating the database was challenging.

Solution

I implemented generic `config` and `elements` fields using Mongoose `Schema.Types.Mixed` so different slide types can store custom data cleanly.

Outcome

Initial database schemas created and connected to MongoDB Atlas.

14/07/2026

Activity

Login and Registration UI Development

Work Completed

Today I focused on building the user authentication interface. I developed the Login page (`/login`) and Signup page (`/register`) using React server and client components. I added responsive layouts, form input fields, password visibility toggles, and client-side validation using state hooks. I also styled the pages using our monochrome design system.

Problems Faced

The login form layout had alignment issues on mobile screen widths below 380px.

Solution

I adjusted flexbox container utility classes and added responsive padding `px-4 sm:px-6` to ensure smooth rendering on smaller screens.

Outcome

Responsive Login and Signup user interface pages built and tested on multiple screen sizes.

15/07/2026

Activity

Resend Email Service Integration

Work Completed

Today I integrated the Resend API to handle transactional email delivery. I created an email service in `apps/backend/src/services/email.ts` to send password reset and email verification links. I built the Forgot Password (`/forgot-password`), Reset Password (`/reset-password`), and Email Verification (`/verify-email`) frontend pages.

Problems Faced

Resend API calls failed during local testing because an API key was not yet set up in local environment variables.

Solution

I added a fallback check in the email service that logs email content to the console in development mode when no API key is present.

Outcome

Transactional email service integrated and tested for password reset and email verification flows.

16/07/2026

Activity

Authentication API Development and JWT Implementation

Work Completed

Today I wrote the backend authentication APIs in `apps/backend/src/routes/auth.ts`. I implemented registration, login, token refresh, logout, and profile update endpoints. I added bcrypt password hashing (12 salt rounds) and configured JWT token issuance. I also wrote the `requireAuth` middleware to protect private routes.

Problems Faced

The refresh token cookie was not being sent back from the client during cross-origin API calls.

Solution

I added `credentials: "include"` to client fetch requests and enabled `credentials: true` in backend CORS options with explicit origin matching.

Outcome

Complete authentication API flow working with secure JWT tokens and HTTP-only cookies.

17/07/2026

Activity

Database Schema Refinement and Validation Rules

Work Completed

I reviewed and updated all database schemas today. I refined the `User`, `Presentation`, `Slide`, `Session`, and `Notification` schemas to enforce strict data types, required fields, and index rules. I added unique indexes for user emails, presentation share IDs, and session codes to optimize database queries.

Problems Faced

Creating unique indexes on sparse fields like `sessionCode` caused duplicate key errors on null values.

Solution

I configured the `sessionCode` schema field with `{ sparse: true, unique: true }` so documents without an active session code are ignored by the index.

Outcome

Database schemas updated with strict validation rules and performance indexes.

18/07/2026

Activity

Dashboard Layout and Navigation Components

Work Completed

Today I designed and built the presenter dashboard layout. I created a collapsible sidebar component containing navigation links for Presentations, Analytics, and Settings. I added a mobile hamburger header, top bar with avatar menu, dark mode toggle, and notification dropdown menu (`NotificationCenter`). I also built metric cards showing presentation counts.

Problems Faced

The sidebar overlay was blocking click events on the main dashboard content on tablet screens.

Solution

I refactored the sidebar Z-index levels and added a backdrop overlay handler that closes the mobile menu when clicking outside.

Outcome

Responsive presenter dashboard layout completed with functional navigation and notification center.

19/07/2026

Activity

Presentation Management Page Development

Work Completed

Today I developed the Presentation Management page (`/presentations`). Presenters can view their list of presentations, search by title, and filter by status (draft, published, archived). I implemented endpoints for creating, duplicating, archiving, and soft-deleting presentations. I also added Azure Blob Storage integration for uploading presentation cover images.

Problems Faced

The search filter was triggering an API request on every keystroke, causing unnecessary server load.

Solution

I added debounced state handling so API requests only fire after the user stops typing for 400 milliseconds.

Outcome

Presentation management page completed with working search, status filters, and CRUD operations.

20/07/2026

Activity

Presentation Builder and Canvas Components

Work Completed

I spent today building the core Presentation Builder interface (`/presentations/[id]/edit`). I created the left slide navigator (filmstrip view), center slide canvas manager, contextual toolbar, and slide theme settings modal. I implemented a debounced auto-save hook (`useAutoSave`) that automatically saves slide changes to the database.

Problems Faced

Rapid slide editing caused out-of-order auto-save API requests when network responses arrived late.

Solution

I added request sequencing and save status indicators ("Saving...", "Saved") to track auto-save states accurately.

Outcome

Presentation Builder shell operational with slide navigation, auto-save, and theme customizers.

21/07/2026

Activity

Socket.IO Real-Time Layer Integration

Work Completed

Today I integrated Socket.IO into the backend server for real-time WebSocket communication. I created room connection logic for presentation live sessions so audience members and presenters can join the same session room. I wrote event handlers for connection, disconnection, and basic message broadcasting.

Problems Faced

WebSocket connections failed when testing cross-origin requests from the Next.js frontend server.

Solution

I configured the Socket.IO server CORS settings to use the dynamic `checkOrigin` helper, allowing connection requests from local and production origins.

Outcome

Socket.IO server integrated and verified for room joins and real-time event broadcasting.

22/07/2026

Activity

Groq AI Service Layer Integration

Work Completed

Today I integrated the Groq SDK into the backend to establish our AI Service Layer. I created prompt helper templates for generating quiz questions and poll choices based on user topics. I wrote backend test endpoints to call Groq's Llama 3 model and parse JSON responses.

Problems Faced

The AI model sometimes returned conversational markdown text instead of pure JSON, causing parsing errors.

Solution

I updated prompt templates with explicit system instructions demanding strict JSON output, and added validation wrappers using Zod schemas.

Outcome

Groq AI service integrated and verified for automated quiz and poll content generation.

23/07/2026

Activity

UI Polish, Responsiveness, and Code Cleaning

Work Completed

Today I focused on cleaning up the frontend codebase and fixing minor UI bugs. I checked page layouts across mobile, tablet, and desktop breakpoints. I updated button hover states, form input focus outlines, and dark mode colors. I also removed unused icon imports and console logs across several files.

Problems Faced

Dark mode styles on certain card components had poor contrast ratios in high-brightness environments.

Solution

I adjusted CSS custom property colors in `globals.css` for `--color-bg-surface` and `--color-text-secondary` to ensure WCAG compliant contrast.

Outcome

Frontend UI polished with improved responsiveness, dark mode contrast, and clean code formatting.

24/07/2026

Activity

Code Review, Documentation Update, and Module Planning

Work Completed

Today I conducted a full code audit of the completed modules. I refactored backend route handlers into separate Controllers and Services to follow Clean Architecture principles. I expanded the `@sentio/shared` package with Zod validation schemas and TypeScript interfaces. Finally, I updated `WorkPlan.md`, `Progress.md`, and `ScrumBook.md` to document all completed architecture and sprint

milestones.

Problems Faced

Consolidating all project documentation while strictly adhering to the 3-file documentation policy required restructuring existing project files.

Solution

I integrated all architecture details, database schemas, API specs, and 20 detailed project modules directly into `WorkPlan.md` and updated `Progress.md` and `ScrumBook.md`.

Outcome

Clean Architecture refactor complete, shared package expanded, and master project documentation synchronized.